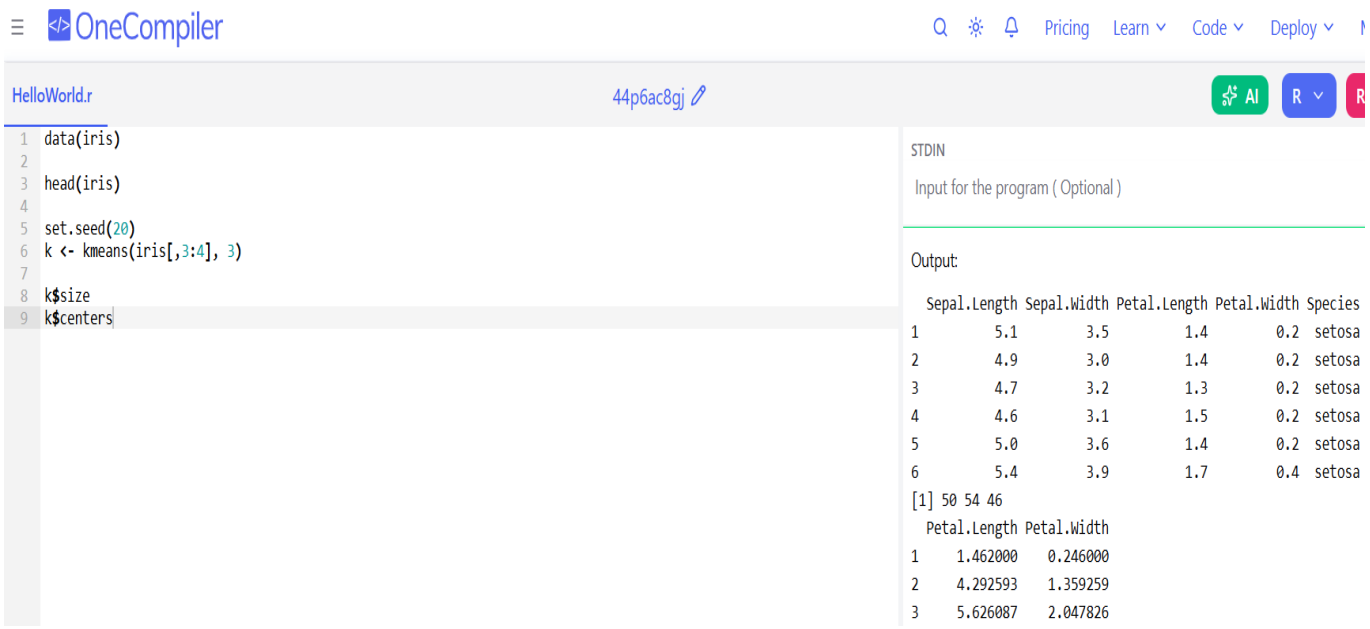
Ex.No.6 — Implementation of Clustering Techniques

Aim

To implement clustering techniques for iris data set.

Program and output:



Algorithm / Procedure

Step 1

Start the R program.

Step 2

Load the iris dataset.

Step 3

Select petal length and petal width features.

Step 4

Choose the number of clusters as K = 3.

Step 5

Initialize random centroids.

Step 6

Apply K-Means clustering algorithm.

Step 7

Assign data points to nearest cluster.

Step 8

Recalculate cluster centroids.

Step 9

Repeat clustering until stable clusters are formed.

Step 10

Display cluster results and tables.

Step 11

Stop the program.

Ex.No.5a — Implement SVM Classification Techniques

Aim

To implement Support Vector Machine (SVM) to find optimum hyper plane (line in 2D, 3D hyper plane) which maximize the margin between two classes.

Program:

```
# Load package

library(e1071)


# Load iris dataset

data(iris)


# Plot dataset

plot(iris)


# Scatter plots

plot(iris$Sepal.Length,

    iris$Sepal.Width,

    col = iris$Species)


plot(iris$Petal.Length,

    iris$Petal.Width,
```

```
    col = iris$Species)

# Split data
s <- sample(150, 100)

train <- iris[s, c("Petal.Length",
                   "Petal.Width",
                   "Species")]

test <- iris[-s, c("Petal.Length",
                   "Petal.Width",
                   "Species")]

# Train SVM model
svmfit <- svm(Species ~ .,
              data = train,
              kernel = "linear")

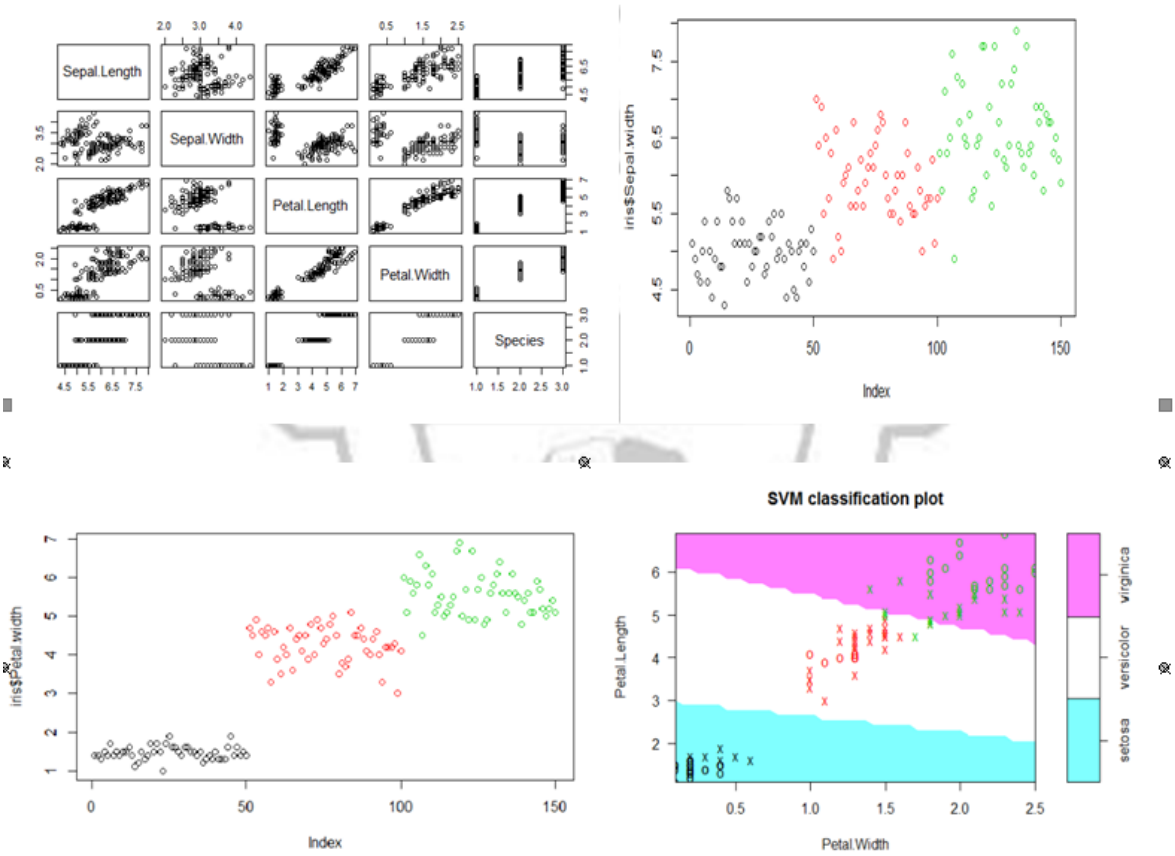
# SVM classification plot
plot(svmfit, train)

# Prediction
p <- predict(svmfit, test)

# Accuracy table
table(p, test$Species)

# Accuracy value
mean(p == test$Species)
```

output:



Ex.No.5a — SVM Classification Technique

Algorithm / Procedure

Step 1

Start the R program.

Step 2

Load the iris dataset.

Step 3

Split the dataset into training and testing data.

Step 4

Select important features for classification.

Step 5

Create the SVM model using training data.

Step 6

Choose linear kernel for classification.

Step 7

Train the SVM classifier.

Step 8

Predict the class labels for testing data.

Step 9

Compare predicted values with actual values.

Step 10

Calculate classification accuracy.

Step 11

Display the output graph and results.

Step 12

Stop the program.

Ex.No.5b — Implement Decision Tree Classification Techniques

Aim

To implement a decision tree used for representing a decision situation visually and showing all those factors within the analysis that are considered relevant to the decision.

Program:

```
# Load packages
library(MASS)
library(rpart)

# Load dataset
head(birthwt)

# Histogram
hist(birthwt$bwt)

# Convert columns to factor
cols <- c("low","race","smoke","ht","ui")
birthwt[cols] <- lapply(birthwt[cols], as.factor)

# Split data
```

```
set.seed(1)
train <- sample(1:nrow(birthwt),
          0.75*nrow(birthwt))

# Decision Tree model
tree <- rpart(low ~ . - bwt,
         data = birthwt[train,],
         method = "class")

# Plot tree
plot(tree)
text(tree)

# Prediction
p <- predict(tree,
        birthwt[-train,],
        type = "class")

# Result table
table(p, birthwt[-train,]$low)
```

Algorithm / Procedure

Step 1

Start the R program.

Step 2

Load the birth weight dataset.

Step 3

Convert required columns into factor type.

Step 4

Split the dataset into training and testing data.

Step 5

Create the Decision Tree model using rpart() function.

Step 6

Select the best attributes for classification.

Step 7

Construct the tree structure.

Step 8

Predict output values using testing data.

Step 9

Compare predicted and actual results.

Step 10

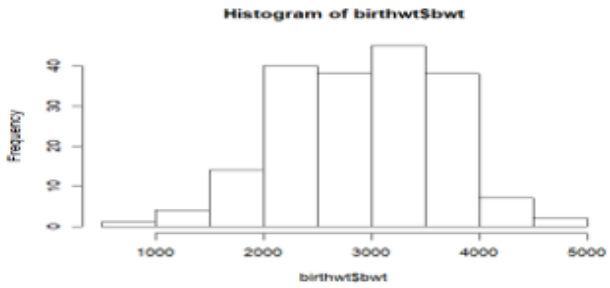
Display the decision tree and output table.

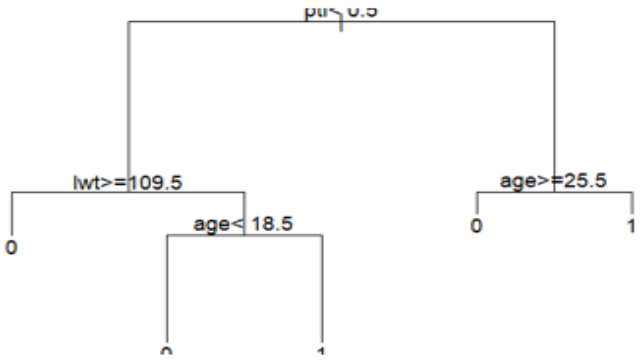
Step 11

Stop the program.

OUTPUT:

	low	age	lwt	race	smoke	ptl	ht	ui	ftv	bwt
85	0	19	182	2	0	0	0	1	0	2523
86	0	33	155	3	0	0	0	0	3	2551
87	0	20	105	1	1	0	0	0	1	2557
88	0	21	108	1	1	0	0	1	2	2594
89	0	18	107	1	1	0	0	1	0	2600
91	0	21	124	3	0	0	0	0	0	2622





Ex.No.4 — Implement Linear and Logistic Regression

Aim

To implement linear and logistic regression for Sales dataset.

Program Code

1. Simple Linear Regression

```
dataset = read.csv("data-marketing-budget-12mo.csv",
                   header = T,
                   colClasses = c("numeric", "numeric", "numeric"))

head(dataset,5)

# Simple Regression

simple.fit = lm(Sales ~ Spend, data = dataset)

summary(simple.fit)
```

```
Call:
lm(formula = Sales ~ Spend, data = dataset)

Residuals:
   Min     1Q Median     3Q    Max
 -3385  -2097    258   1726   3034

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 1383.4714  1255.2404   1.102    0.296
Spend         10.6222     0.1625  65.378 1.71e-14 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2313 on 10 degrees of freedom
Multiple R-squared:  0.9977, Adjusted R-squared:  0.9974
F-statistic:  4274 on 1 and 10 DF,  p-value: 1.707e-14
```

---

2. Multiple Linear Regression

```
multi.fit = lm(Sales ~ Spend + Month, data = dataset)

summary(multi.fit)
```

```
Call:
lm(formula = Sales ~ Spend + Month, data = dataset)

Residuals:
     Min       1Q   Median       3Q      Max
-1793.73 -1558.33    -1.73  1374.19  1911.58

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -567.6098  1041.8836  -0.545  0.59913
Spend         10.3825     0.1328  78.159 4.65e-14 ***
Month        541.3736   158.1660   3.423  0.00759 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1607 on 9 degrees of freedom
Multiple R-squared:  0.999, Adjusted R-squared:  0.9988
F-statistic:  4433 on 2 and 9 DF,  p-value: 3.368e-14
```

---

3. Logistic Regression

```
# selects some column from mtcars

input <- mtcars[, c("am", "cyl", "hp", "wt")]

print(head(input))

input <- mtcars[, c("am", "cyl", "hp", "wt")]

am.data = glm(formula = am ~ cyl + hp + wt,
        data = input,
        family = binomial)

print(summary(am.data))
```

```
> print(head(input))
                  am cyl  hp    wt
Mazda RX4          1   6 110 2.620
Mazda RX4 Wag      1   6 110 2.875
Datsun 710         1   4  93 2.320
Hornet 4 Drive     0   6 110 3.215
Hornet Sportabout  0   8 175 3.440
Valiant            0   6 105 3.460
```

```
Call:
glm(formula = am ~ cyl + hp + wt, family = binomial, data = input)

Deviance Residuals:
     Min        1Q    Median        3Q       Max
-2.17272  -0.14907  -0.01464   0.14116   1.27641

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) 19.70288    8.11637   2.428   0.0152 *
cyl          0.48760    1.07162   0.455   0.6491
hp           0.03259    0.01886   1.728   0.0840 .
wt          -9.14947    4.15332  -2.203   0.0276 *
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 43.2297  on 31  degrees of freedom
Residual deviance:  9.8415  on 28  degrees of freedom
AIC: 17.841

Number of Fisher Scoring iterations: 8
```

Algorithm / Procedure

Step 1

Start the R program.

Step 2

Load the sales dataset into R.

Step 3

Perform Simple Linear Regression using Sales and Spend.

Step 4

Fit the regression model using lm() function.

Step 5

Display the regression summary and prediction results.

Step 6

Perform Multiple Linear Regression using Sales, Spend, and Month.

Step 7

Fit the multiple regression model.

Step 8

Display the multiple regression output.

Step 9

Load the dataset for Logistic Regression.

Step 10

Apply logistic regression using glm() function.

Step 11

Predict categorical output values.

Step 12

Display the logistic regression result.

Step 13

Stop the program.

Ex.No.7 — Implementation of Visualize Data using Any Plotting Framework

Aim

To implement data visualization to provide an efficient graphical display for summarizing and reasoning about quantitative information.

Program:

```
# Load Libraries
library(RColorBrewer)
library(corrplot)
library(ggplot2)

# ---------- 1. HISTOGRAM ----------
data(VADeaths)

par(mfrow=c(2,3))

hist(VADeaths,
   col=brewer.pal(3,"Set3"),
   main="Set3 colors")

hist(VADeaths,
   col=brewer.pal(3,"Set2"),
   main="Set2 colors")
```

```
hist(VADeaths,
    col=brewer.pal(3,"Set1"),
    main="Set1 colors")

hist(VADeaths,
    col=brewer.pal(8,"Set3"),
    main="Set3 8 colors")

hist(VADeaths,
    col=brewer.pal(8,"Greys"),
    main="Greys colors")

hist(VADeaths,
    col=brewer.pal(8,"Greens"),
    main="Greens colors")

# ---------- 2. LINE CHART ----------
data(AirPassengers)

plot(AirPassengers,
    type="l",
    main="Line Chart")

# ---------- 3. BAR CHART ----------
data(iris)

barplot(iris$Sepal.Length,
       col="red",
       main="Bar Chart")

# ---------- 4. BOX PLOT ----------
```

```
par(mfrow=c(2,2))

boxplot(iris$Sepal.Length,
      col="red")

boxplot(iris$Sepal.Length ~ iris$Species,
      col="orange")

boxplot(iris$Sepal.Length ~ iris$Species,
      col=heat.colors(3))

boxplot(iris$Petal.Length ~ iris$Species,
      col=topo.colors(3))

# ---------- 5. SCATTER PLOT ----------
plot(iris$Sepal.Length,
   iris$Sepal.Width,
   main="Scatter Plot")

plot(iris$Petal.Length,
   iris$Petal.Width,
   col=iris$Species)

# ---------- 6. HEAT MAP ----------
x <- matrix(rnorm(100),10)

heatmap(x)

# ---------- 7. CORRELOGRAM ----------
corr_matrix <- cor(mtcars)
```

```
corrplot(corr_matrix)


corrplot(corr_matrix,
         method="number",
         type="lower")


# ---------- 8. AREA CHART ----------
ggplot(airquality,
       aes(Day, Wind)) +
geom_area() +
labs(title="Area Chart")
```

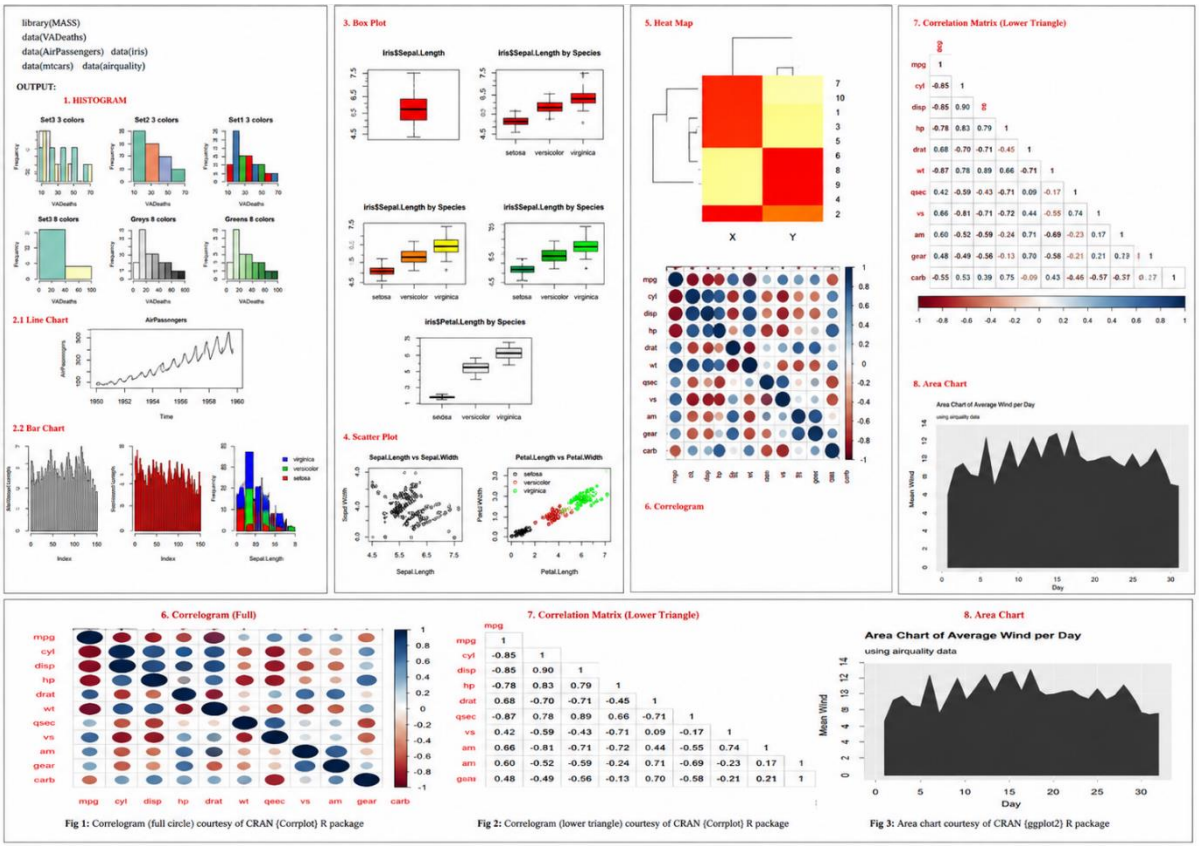


Fig 1: Correlogram (full circle) courtesy of CRAN {Corrplot} R package

Fig 2: Correlogram (lower triangle) courtesy of CRAN {Corrplot} R package

Fig 3: Area chart courtesy of CRAN {ggplot2} R package

Algorithm / Procedure

Step 1

Start the R program.

Step 2

Load the required datasets and libraries.

Step 3

Create Histogram for frequency distribution.

Step 4

Create Line Chart to show trends over time.

Step 5

Create Bar Chart for comparing values.

Step 6

Create Box Plot to show data spread.

Step 7

Create Scatter Plot to show relationship between variables.

Step 8

Create Heat Map for visualizing data intensity.

Step 9

Create Correlogram for correlation analysis.

Step 10

Create Area Chart for continuous data visualization.

Step 11

Display all visualization outputs.

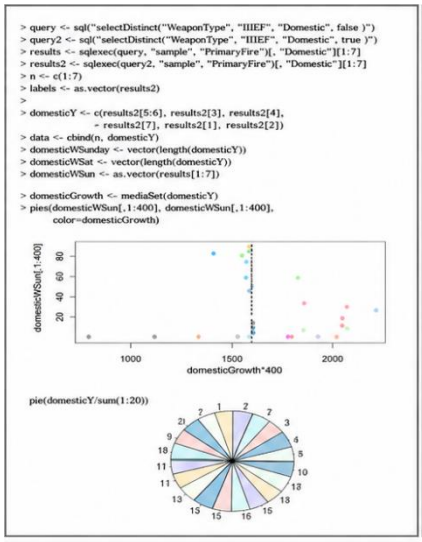
Step 12

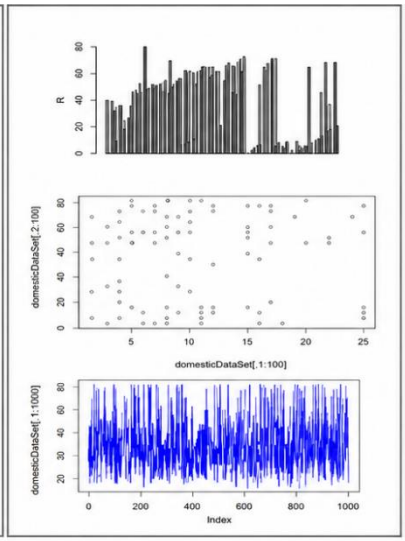
Stop the program.

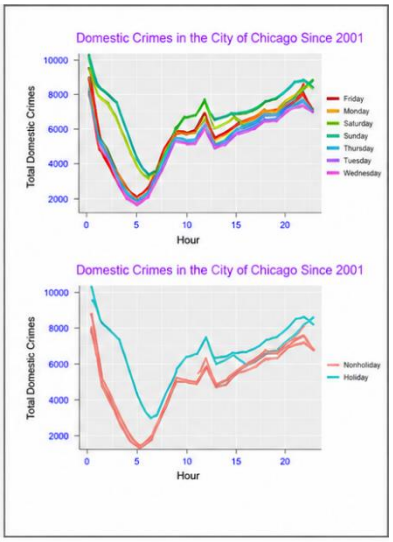
Ex.No.8 — Implement an application that stores big data in MongoDB using R

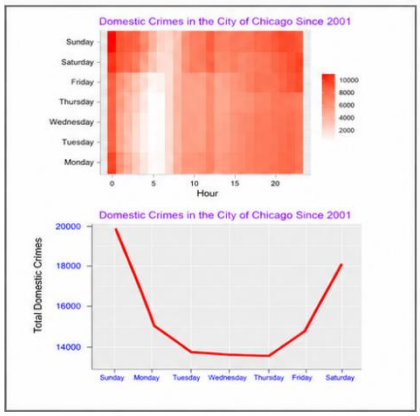
Aim

To implement an application that stores big data in MongoDB using R.









Algorithm / Procedure

Step 1

Start the R program.

Step 2

Install and load MongoDB related libraries.

Step 3

Load the crime dataset into R.

Step 4

Create connection between R and MongoDB.

Step 5

Create database and collection.

Step 6

Insert crime dataset into MongoDB collection.

Step 7

Retrieve records from MongoDB using queries.

Step 8

Perform filtering and aggregation operations.

Step 9

Analyze domestic crime data.

Step 10

Create plots and graphs for visualization.

Step 11

Display MongoDB query results and charts.

Step 12

Stop the program.

Program:

```
# Libraries
library(ggplot2)

# Sample Data
x <- 1:24
y <- sample(2000:10000,24)

# 1. Bar Plot
barplot(y,
      col="grey",
      main="Bar Plot")

# 2. Scatter Plot
plot(x, y,
   main="Scatter Plot")

# 3. Line Plot
plot(y,
   type="l",
   col="blue",
```

```
    main="Line Plot")

# 4. Pie Chart
pie(c(10,20,30,40),
   col=rainbow(4))

# 5. Heat Map
m <- matrix(rnorm(100),10)

heatmap(m)

# 6. Domestic Crime Line Graph
df <- data.frame(Hour=x,
                 Crimes=y)

ggplot(df,
       aes(Hour, Crimes)) +
geom_line(color="red",
          size=1.5) +
ggtitle("Domestic Crimes in Chicago")

# 7. Weekend vs Weekday Graph
df$Type <- ifelse(df$Hour>12,
                  "Weekend",
                  "Weekday")

ggplot(df,
       aes(Hour, Crimes,
       color=Type)) +
geom_line(size=1.5)
```

```
# 8. Heatmap Style Plot

ggplot(df,

    aes(Hour, Type,

    fill=Crimes)) +

geom_tile()
```

Ex.No.3 — Implement a MR program that process a weather dataset

Aim

To implement a MR (MapReduce) program that processes a weather dataset.

Program:

1. Mapper.java

```
import java.io.*;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Mapper;

public class Mapper1 extends Mapper<LongWritable,Text,Text,IntWritable>
{
  public void map(LongWritable k, Text v, Context c)
  throws IOException, InterruptedException
  {
    String s = v.toString();

    c.write(
    new Text(s.substring(15,19)),
    new IntWritable(
    Integer.parseInt(s.substring(87,92).trim()))
    );
  }
}
```

---

2. Reducer.java

```
import java.io.*;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Reducer;

public class Reducer1 extends Reducer<Text,IntWritable,Text,IntWritable>
{
  public void reduce(Text k,
            Iterable<IntWritable> v,
            Context c)
```

```
  throws IOException, InterruptedException
  {
    int sum=0,count=0;

    for(IntWritable i:v)
    {
      sum += i.get();
      count++;
    }

    c.write(k,new IntWritable(sum/count));
  }
}
```

---

3. Driver.java

```
import org.apache.hadoop.conf.*;
import org.apache.hadoop.fs.*;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.*;
import org.apache.hadoop.mapreduce.lib.output.*;

public class Driver
{
  public static void main(String[] a)
  throws Exception
  {
    Job j = Job.getInstance(new Configuration(),"Avg");

    j.setJarByClass(Driver.class);

    j.setMapperClass(Mapper1.class);

    j.setReducerClass(Reducer1.class);

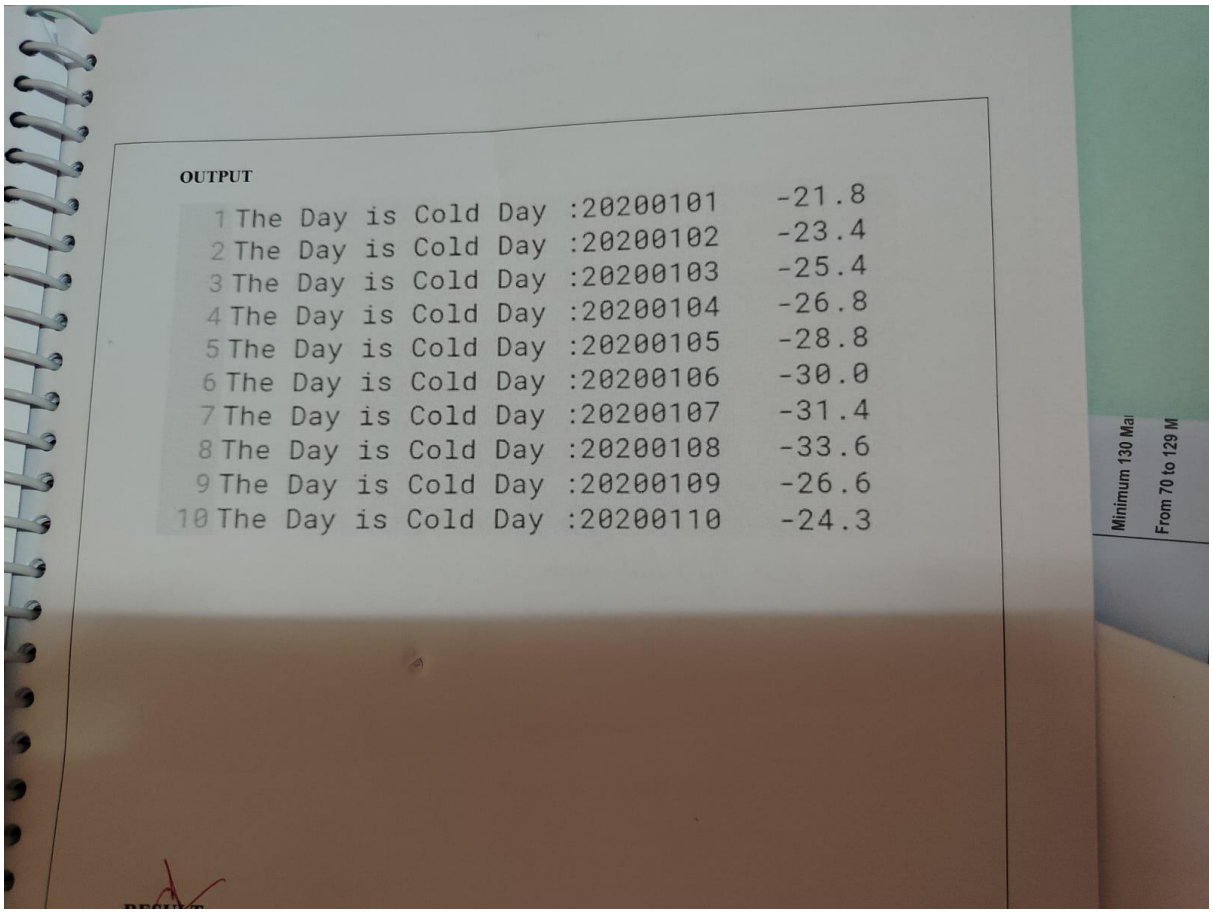
    j.setOutputKeyClass(Text.class);

    j.setOutputValueClass(IntWritable.class);

    FileInputFormat.addInputPath(j,new Path(a[0]));

    FileOutputFormat.setOutputPath(j,new Path(a[1]));

    System.exit(j.waitForCompletion(true)?0:1);
  }
}
```



OUTPUT

```
 1 The Day is Cold Day :20200101    -21.8
 2 The Day is Cold Day :20200102    -23.4
 3 The Day is Cold Day :20200103    -25.4
 4 The Day is Cold Day :20200104    -26.8
 5 The Day is Cold Day :20200105    -28.8
 6 The Day is Cold Day :20200106    -30.0
 7 The Day is Cold Day :20200107    -31.4
 8 The Day is Cold Day :20200108    -33.6
 9 The Day is Cold Day :20200109    -26.6
10 The Day is Cold Day :20200110    -24.3
```

Algorithm / Procedure

Step 1

Start the Hadoop MapReduce program.

Step 2

Load the weather dataset into HDFS.

Step 3

Create the Mapper class to read weather records.

Step 4

Extract year and temperature values from the dataset.

Step 5

Check whether the temperature value is valid.

Step 6

Send year and temperature as key-value pairs to Reducer.

Step 7

Reducer receives all temperature values of the same year.

Step 8

Calculate the average temperature for each year.

Step 9

Store the final output in Hadoop output folder.

Step 10

Stop the program.